

Enhancing Presolve in Mixed Integer Programming by Combining Probing and Dual Fixing

Zhao-Wei Wang

Institute of Computational Mathematics and Scientific/Engineering Computing
Academy of Mathematics and Systems Science
Chinese Academy of Sciences, China

Joint work with Prof.Wei-Kun Chen and Prof.Yu-Hong Dai

ASA2026, Fuzhou

April 17, 2026

CONTENTS

- ① Preliminary
- ② Integration of dual fixing into probing
- ③ Improved conditions for applying dual fixing
- ④ Numerical experiments
- ⑤ Conclusions

CONTENTS

- ① Preliminary
- ② Integration of dual fixing into probing
- ③ Improved conditions for applying dual fixing
- ④ Numerical experiments
- ⑤ Conclusions

Mixed integer programming

Given

- ▶ a matrix $A \in \mathbb{R}^{m \times n}$
- ▶ vectors $c \in \mathbb{R}^n$, $\ell \in (\mathbb{R} \cup \{-\infty\})^n$, $u \in (\mathbb{R} \cup \{+\infty\})^n$
- ▶ vectors $b \in \mathbb{R}$
- ▶ variables $x \in \mathbb{R}^n$ with $x_j \in \mathbb{Z}$ for $j \in \mathcal{J} \subseteq \mathcal{N} := \{1, \dots, n\}$

then the optimization problem $\text{MIP}(A, b, c, \ell, u, x, \mathcal{J})$ can be written as

$$\begin{array}{ll} \min & c^\top x \\ \text{s.t.} & Ax \leq b \\ & \ell \leq x \leq u, \ x_{\mathcal{J}} \in \mathbb{Z}^{|\mathcal{J}|}. \end{array} \quad (\text{MIP})$$

We denote the feasible domain of (MIP) as

$$\mathcal{X} = \{x \in \mathbb{R}^{|\mathcal{N}|} : Ax \leq b, \ \ell \leq x \leq u, \ x_{\mathcal{J}} \in \mathbb{Z}^{|\mathcal{J}|}\}.$$

Presolve in MIP

Presolve contains a set of routines designed to **eliminate redundant information** and **strengthen the formulation**, with the goal of accelerating the subsequent branch-and-cut process.

- ▶ [Achterberg et al. \[2019\]](#): “... **presolve** together with cutting plane techniques are by far **the most important individual tools** contributing to the power of modern MIP solvers.”
- ▶ When presolve is disabled, Gurobi’s performance degrades by a factor of **9**, CPLEX’s performance degrades by a factor of **11.4** [[Achterberg and Wunderling, 2013](#)].

In this talk, we focus on **dual fixing and probing**.

Dual fixing: an illustrative example

Consider the following MIP:

$$\begin{array}{ll} \min & -x_2 \\ \text{s.t.} & x_1 + x_2 \leq 1 \\ & 0 \leq x_1 \leq 1, 0 \leq x_2 \leq 1 \end{array} \quad (1)$$

- ▶ As the **objective coefficient** of x_1 is **nonnegative**, and all constraints (without bound constraints) has **nonnegative coefficients** of x_1 , $x_1 = 0$ can be set in problem (1).
- ▶ Variable x_1 and constraint $x_1 + x_2 \leq 1$ can be removed from problem (1).

Dual fixing: basic approach [Achterberg, 2007]

Dual fixing sets a variable to its upper bound (respectively, lower bound) if

- (i) increasing (respectively, decreasing) the variable's value makes **all the constraints other than its bound constraints** in the problem **less tight**, and
- (ii) the variable has a **nonpositive (respectively, nonnegative) objective coefficient**.

Dual fixing is an **optimality-based** presolve technique.

Dual fixing: basic approach [Achterberg, 2007]

Dual fixing considers the following two cases

(DF1) $c_j \geq 0$, $a_{ij} \geq 0$, $\forall i \in \mathcal{M}$, and

(DF2) $c_j \leq 0$, $a_{ij} \leq 0$, $\forall i \in \mathcal{M}$.

Consider the case (DF1),

- ▶ if $\ell_j > -\infty$, x_j can be fixed to its lower bound;
- ▶ if $\ell_j = -\infty$ and $c_j > 0$, the problem is unbounded;
- ▶ if $\ell_j = -\infty$ and $c_j = 0$, we can remove all constraints containing x_j [Mészáros and Suhl, 2003].

For variable x_j , $j \in \mathcal{N}$ in (MIP), define the **down-locks** and **up-locks**

$$\mathcal{C}_j^- = \{i \in \mathcal{M} \mid a_{ij} < 0\},$$

$$\mathcal{C}_j^+ = \{i \in \mathcal{M} \mid a_{ij} > 0\}.$$

Probing: an illustrative example

Consider the following MIP:

$$x_1 + x_2 + x_3 \leq 3y, \quad (2)$$

$$x_1 \geq y, \quad (3)$$

$$x_1, x_2, x_3, y \in \{0, 1\}, \quad (4)$$

and probe on y , we obtain

- ▶ Fixing $y = 0 \stackrel{(2)}{\Rightarrow} x_1 = x_2 = x_3 = 0$.
- ▶ Fixing $y = 1 \stackrel{(3)}{\Rightarrow} x_1 = 1$.
- ▶ Variable substitution: $x_1 = y$.
- ▶ Variable implications: $x_2 \leq y, x_3 \leq y$.

Probing: basic approach [Savelsbergh, 1994]

Probing **tentatively** set some binary variables to 0 or 1, apply (linear constraint based) **domain propagation techniques** to derive better variable bounds, and extract useful information such as **stronger variable implications** and **better global variable bounds**.

Probing is a **feasibility-based** presolve technique.

Probing: basic approach [Savelsbergh, 1994]

Let $x_k \in \{0, 1\}$ be a binary variable. Denote $\ell_j^0(u_j^0)$ and $\ell_j^1(u_j^1)$ be the lower (upper) bounds of x_j , under condition $x_k = 0$ and 1, respectively.

- R1. If one of the fixings of x_k leads to an infeasible problem, x_k can be fixed to the opposite value.
- R2. If $\ell_j^0 = u_j^0$ and $\ell_j^1 = u_j^1$, x_j can be aggregated as $x_j := \ell_j^0 + (\ell_j^1 - \ell_j^0)x_k$.
- R3. $\ell'_j := \min\{\ell_j^0, \ell_j^1\}$ and $u'_j := \max\{u_j^0, u_j^1\}$ are valid global bounds of x_j .
- R4. Variable implications between x_k and x_j :

$$x_j \geq (\ell_j^1 - \ell_j^0)x_k + \ell_j^0, \quad x_j \leq (u_j^1 - u_j^0)x_k + u_j^0. \quad (5)$$

Motivation of this talk

However to combine dual fixing and probing?

CONTENTS

- ① Preliminary
- ② Integration of dual fixing into probing
- ③ Improved conditions for applying dual fixing
- ④ Numerical experiments
- ⑤ Conclusions

An illustrative example

Consider the following example:

$$\begin{array}{ll}\min & x_1 + x_2 \\ \text{s.t.} & x_1 - x_2 \leq 0 \\ & x_1, x_2 \in \{0, 1\}.\end{array}$$

- ▶ Probing on $x_1 = 1$, x_2 can be fixed to 1 by **domain propagation**.
- ▶ Probing on $x_1 = 0$, constraint $x_1 - x_2 \leq 0$ becomes redundant and the problem reduces to

$$\begin{array}{ll}\min & x_1 + x_2 \\ \text{s.t.} & x_1 = 0, x_2 \in \{0, 1\},\end{array}$$

and we can fix $x_2 = 0$ by **dual fixing**.

- ▶ Variable substitution: $x_2 = x_1$.

Integration of dual fixing into probing

- In probing, we fix $x_k = v \in \{0, 1\}$ and apply domain propagation on the following modified subproblem

$$\begin{array}{ll} \min & c^\top x \\ \text{s. t.} & Ax \leq b \\ & \ell \leq x \leq u, \ x_J \in \mathbb{Z}^{|J|}, \ J \subseteq \mathcal{N}, \ \mathbf{x}_k = \mathbf{v} \end{array} \quad (\text{MIP-v})$$

to derive tighter variable bounds ℓ', u' valid for (MIP-v).

Integration of dual fixing in probing

- These tighter variable bounds ℓ', u' may render some constraints **redundant**, leading to the following MIP (equivalent to (MIP-v))

$$\begin{array}{ll} \min & c^\top x \\ \text{s. t.} & \mathbf{A}_{\mathcal{R}} \mathbf{x} \leq \mathbf{b}_{\mathcal{R}} \\ & \ell' \leq x \leq u', \ x_{\mathcal{J}} \in \mathbb{Z}^{|\mathcal{J}|}, \ \mathcal{J} \subseteq \mathcal{N}, \ \mathbf{x}_{\mathcal{K}} = \mathbf{v}, \end{array} \quad (\text{MIP-}\mathcal{R})$$

where $\mathcal{R} \subseteq \mathcal{M}$ is the index set of non-redundant constraints.

- **Dual fixing** can be applied on (MIP- $\mathcal{R}$) to derive tighter variable bounds $\ell^v (\geq \ell')$, $u^v (\leq u')$, as the down-locks and up-locks for variables in (MIP- $\mathcal{R}$) could become smaller

$$\begin{aligned} \bar{\mathcal{C}}_j^- &= \{i \in \mathcal{R} \mid a_{ij} < 0\} \leq \mathcal{C}_j^-, \\ \bar{\mathcal{C}}_j^+ &= \{i \in \mathcal{R} \mid a_{ij} > 0\} \leq \mathcal{C}_j^+. \end{aligned}$$

Algorithm Framework

Algorithm 1 Dual fixing augmented probing

Input: A binary variable x_k to be probed on, A , b , c , ℓ , u , $\mathcal{I}$.

- 1: **for** $v = 0, 1$ **do**
 - 2: Set $x_k = v$ in problem (MIP), obtaining subproblem (MIP- v).
 - 3: **repeat**
 - 4: Call **domain propagation presolver** on (MIP- v) to obtain bounds ℓ' , u' .
 - 5: Call **redundant row presolver** to compute the set $\mathcal{R}$ of non-redundant
 - 6: constraints.
 - 7: Call **dual fixing presolver** on (MIP- $\mathcal{R}$) to obtain tighter bounds ℓ^v , u^v .
 - 8: **until** no reduction is found.
 - 9: **end for**
 - 10: Extract reductions using R1-R4.
-

Step 5 and 6 in Algorithm 1 can be implemented in $\mathcal{O}(\text{nnz})$, where nnz is the number of non-zeros in the constraint matrix A .

Inconsistent optimality based reductions - 1/2

Comment from the reviewer:

... That's a rather small subset, so I'm less fussed now. It's not unlikely that these are simply bugs in HiGHS. I'd say that [neos-3656078-kumeu](#) looks like it's exposing a bug that was potentially added by your changes though.

When probing on **multiple binary variables** $\{x_k\}$, special care must be taken on the implications (5):

$$x_j \geq (\ell_j^1 - \ell_j^0) x_k + \ell_j^0, \quad x_j \leq (u_j^1 - u_j^0) x_k + u_j^0, \quad (5)$$

if the tightened bounds of x_j are reasoned from applying **dual fixing** on variable x_t , $t \in \mathcal{N} \setminus \{k\}$ with $\mathbf{c}_t = \mathbf{0}$ in step 6 of Algorithm 1.

Inconsistent optimality based reductions - 1/2

- ▶ If $c_t > 0$ or $c_t < 0$, then the implications of variable x_t in (5) obtained by dual fixing will **not remove any optimal solution** of problem (MIP).
- ▶ However, if $c_t = 0$, then the implications of variable x_j in (5) obtained by dual fixing **may exclude some optimal solutions** of problem (MIP).
- ▶ Such implications, could be **inconsistent** with the subsequent reductions derived by probing on other variables, as simultaneously adding them into problem (MIP) **may exclude all optimal solutions**.

Example

Consider the following example:

$$\begin{array}{ll}\min & -x_1 - x_2 \\ \text{s.t.} & x_1 + x_2 + x_3 \geq 1, \\ & x_3 - x_2 \leq 0, \\ & x_1, x_2, x_3 \in \{0, 1\}.\end{array}$$

- ▶ Probing on $x_1 = 1$, constraint $x_1 + x_2 + x_3 \geq 1$ becomes redundant, from $\mathbf{c_3 = 0}$ we can fix $x_3 = 0$ by dual fixing, obtaining implication $\mathbf{x_3 \leq 1 - x_1}$.
- ▶ Probing on $x_2 = 1$, constraint $x_3 - x_2 \leq 0$ becomes redundant. From $\mathbf{c_3 = 0}$ we can fix $x_3 = 1$ by dual fixing, obtaining implication $\mathbf{x_2 \leq x_3}$.
- ▶ Probing on $x_2 = 0$, we can fix $x_3 = 0$ by domain propagation, obtaining implication $x_3 \leq x_2$.
- ▶ Therefore, we obtain the variable aggregation $\mathbf{x_3 = x_2}$.

Example - continue

$$\begin{array}{ll}\min & -x_1 - x_2 \\ \text{s.t.} & x_1 + x_2 + x_3 \geq 1, \\ & x_3 - x_2 \leq 0, \\ & x_1, x_2, x_3 \in \{0, 1\}.\end{array}$$

- ▶ All the optimal solutions are $(1, 1, 0)$ and $(1, 1, 1)$.
- ▶ $x_3 \leq 1 - x_1$ excludes the optimal solution $(1, 1, 1)$ but is valid, as the optimal solution $(1, 1, 0)$ is still retained after adding it.
- ▶ $x_3 = x_2$ excludes the optimal solution $(1, 1, 0)$ but is valid, as the optimal solution $(1, 1, 1)$ is still retained after adding it.
- ▶ However, $x_3 \leq 1 - x_1$ is **inconsistent with $x_3 = x_2$** as simultaneously adding it and $x_3 = x_2$ will **eliminate all the two optimal solutions** $(1, 1, 0)$ and $(1, 1, 1)$.

Inconsistent optimality based reductions - 1/2

To avoid such inconsistency, we

- ▶ **temporarily store the bound changes** of ℓ^v and ℓ (respectively, u^v and u where $v \in \{0, 1\}$) for which the reasoning comes from dual fixing of variable x_t with $c_t = 0$;
- ▶ use them, together with other bound changes, to **perform the global reductions R1, R2, and R4**;
- ▶ **disregard** them right after probing on variable x_k .

Inconsistent optimality based reductions - 2/2

In the implementation of the probing in MIP solvers (e.g., HiGHS),

- ▶ **variable fixing and bound tightening** in R1 and R4 are performed **right after** probing on the current variable x_k ,
- ▶ **variable substitutions** in R2 are **delayed** until all candidate variables in the current round are probed on.

The delayed fashion could **still lead to inconsistency**.

Example

Consider the following example:

$$\begin{array}{ll}\min & -x_1 - x_2 \\ \text{s.t.} & x_1 + x_2 + x_3 \geq 1, \\ & x_3 - x_2 \leq 0, \\ & x_3 \geq 1 - x_1,^* \\ & x_1, x_2, x_3 \in \{0, 1\}.\end{array}$$

- ▶ All the optimal solutions are still $(1, 1, 0)$ and $(1, 1, 1)$.
- ▶ Probing on x_1 and using R2, we obtain the variable substitution $x_3 = 1 - x_1$.
- ▶ Probing on x_2 and using R2, we obtain the variable substitution $x_3 = x_2$.

Example - continue

$$\begin{array}{ll}\min & -x_1 - x_2 \\ \text{s.t.} & x_1 + x_2 + x_3 \geq 1, \\ & x_3 - x_2 \leq 0, \\ & x_3 \geq 1 - x_1, \\ & x_1, x_2, x_3 \in \{0, 1\}.\end{array}$$

- ▶ However, the two variable substitutions $x_3 = 1 - x_1$ and $x_3 = x_2$ are **inconsistent**, as simultaneously adding both will **eliminate all the two optimal solutions** $(1, 1, 0)$ and $(1, 1, 1)$.
- ▶ If the variable substitution $x_3 = 1 - x_1$ is applied **right after** probing on x_1 , we obtain the reduced problem

$$\min\{x_3 - x_2 - 1 \mid x_3 - x_2 \leq 0, x_2, x_3 \in \{0, 1\}\},$$

for which probing on variable x_2 will **no longer** yield the variable aggregation $x_3 = x_2$.

Inconsistent optimality based reductions - 2/2

To further avoid the **inconsistency** incurred by the **delayed variable substitution**,

- ▶ we assign a fixed **“direction”** for variable x_t with $c_t = 0$ when performing dual fixing.
- ▶ This is equivalent to a **perturbation** on the objective coefficient of variable x_t with $c_t = 0$.

CONTENTS

- ① Preliminary
- ② Integration of dual fixing into probing
- ③ Improved conditions for applying dual fixing
- ④ Numerical experiments
- ⑤ Conclusions

Dual fixing: basic approach [Achterberg, 2007]

$$\mathcal{X} = \{x \in \mathbb{R}^{|\mathcal{N}|} : Ax \leq b, \ell \leq x \leq u, x_j \in \mathbb{Z}^{|\mathcal{J}|}\}.$$

Dual fixing considers the following two cases

(DF1) $c_j \geq 0, a_{ij} \geq 0, \forall i \in \mathcal{M}$, and

(DF2) $c_j \leq 0, a_{ij} \leq 0, \forall i \in \mathcal{M}$.

Consider the case (DF1),

- ▶ if $\ell_j > -\infty$, x_j can be fixed to its lower bound;
- ▶ if $\ell_j = -\infty$ and $c_j > 0$, the problem is unbounded;
- ▶ if $\ell_j = -\infty$ and $c_j = 0$, we can remove all constraints containing x_j .

Limitations for classic dual fixing

Motivation

- ▶ The conditions (DF1) and (DF2) highly **depend on A** to characterize the feasible region $\mathcal{X}$ of (MIP).

Solution

- ▶ Directly take the **feasible region $\mathcal{X}$** into account rather than **a specific constraint matrix**.

Upper bound reachable

Definition 1

A variable x_j is **upper bound reachable** with respect to $\mathcal{X}$ if setting $x_j = u_j$ in **any feasible solution** x of $\mathcal{X}$ yields another **feasible solution**, that is,

$$x = (x_1, \dots, x_j, \dots, x_n) \in \mathcal{X} \Rightarrow \bar{x} = (x_1, \dots, u_j, \dots, x_n) \in \mathcal{X}.$$

When $u_j = +\infty$, this means that $\bar{x} = (x_1, \dots, a, \dots, x_n) \in \mathcal{X}$ holds for any $a \geq x_j$.

Lower bound reachable

Definition 2

A variable x_j is **lower bound reachable** with respect to $\mathcal{X}$ if setting $x_j = \ell_j$ in **any feasible solution** x of $\mathcal{X}$ yields another **feasible solution**, that is,

$$x = (x_1, \dots, x_j, \dots, x_n) \in \mathcal{X} \Rightarrow \bar{x} = (x_1, \dots, \ell_j, \dots, x_n) \in \mathcal{X}.$$

When $\ell_j = -\infty$, this means that $\bar{x} = (x_1, \dots, a, \dots, x_n) \in \mathcal{X}$ holds for any $a \leq x_j$.

Improved conditions for applying dual fixing

It is simple to see that if

(IDF1) $c_j \geq 0$ and x_j is lower bound reachable, or

(IDF2) $c_j \leq 0$ and x_j is upper bound reachable,

then dual fixing can be applied to fix variables to lower or upper bounds.

An illustrative example

Consider the following example

$$\begin{array}{ll}\min & -x_1 - 2x_2 + x_3 \\ \text{s.t.} & 0 \leq 8x_1 + 4x_2 - x_3 \leq 8, \\ & x_1, x_2, x_3 \in \{0, 1\}.\end{array}$$

- ▶ $|\mathcal{C}_3^-| = 1$, dual fixing cannot be applied to fix $x_3 = 0$.
- ▶ However, as

$$\begin{aligned}\mathcal{X} &= \{(x_1, x_2, x_3) \in \{0, 1\}^3 \mid 0 \leq 8x_1 + 4x_2 - x_3 \leq 8\} \\ &= \{(1, 0, \mathbf{1}), (1, 0, \mathbf{0}), (0, 1, \mathbf{1}), (0, 1, \mathbf{0}), (0, 0, \mathbf{0})\},\end{aligned}$$

x_3 is **lower bound reachable**, and by $c_3 = 1 \geq 0$, condition (IDF1) is satisfied, enabling to apply dual fixing to fix $x_3 = 0$.

Determining lower or upper bound reachable

Theorem 3

Let $o_j^{-,r}$ and $o_j^{+,r}$ be defined as

$$o_j^{-,r} := \max_{x_j > \ell_j} \left\{ \sum_{s \in \mathcal{N} \setminus \{j\}} a_{rs} x_s \mid x \in \mathcal{X} \right\}, \quad \forall j \in \mathcal{N}, r \in \mathcal{M} \text{ with } a_{rj} < 0, \quad (6)$$

$$o_j^{+,r} := \max_{x_j < u_j} \left\{ \sum_{s \in \mathcal{N} \setminus \{j\}} a_{rs} x_s \mid x \in \mathcal{X} \right\}, \quad \forall j \in \mathcal{N}, r \in \mathcal{M} \text{ with } a_{rj} > 0. \quad (7)$$

Then for $j \in \mathcal{N}$,

- (i) x_j is lower bound reachable if and only if $o_j^{-,r} + a_{rj} \ell_j \leq b_r$ holds for all $r \in \mathcal{M}$ with $a_{rj} < 0$;
- (ii) x_j is upper bound reachable if and only if $o_j^{+,r} + a_{rj} u_j \leq b_r$ holds for all $r \in \mathcal{M}$ with $a_{rj} > 0$.

An illustrative example

Consider the following example again

$$\begin{array}{ll}\min & -x_1 - 2x_2 + x_3 \\ \text{s.t.} & 0 \leq 8x_1 + 4x_2 - x_3 \leq 8, \\ & x_1, x_2, x_3 \in \{0, 1\}.\end{array}$$

- ▶ As only the constraint $8x_1 + 4x_2 - x_3 \leq 8$ involves variable x_3 with a **negative coefficient**, we only need to consider the optimization problem

$$o_3^{-,2} = \max_{x_3 > 0} \{8x_1 + 4x_2 \mid 0 \leq 8x_1 + 4x_2 - x_3 \leq 8, x_1, x_2, x_3 \in \{0, 1\}\}.$$

- ▶ $o_3^{-,2} = 8 \Rightarrow o_3^{-,2} + (-1)\ell_3 = 8 \leq 8$, then x_3 must be **lower bound reachable**.

Determining lower or upper bound reachable

- ▶ Solving **at most m** optimization problems to obtain $\{o_j^{-,r}\}$ and $\{o_j^{+,r}\}$ for fixing a single variable x_j could still be **computational demanding**.
- ▶ $\{o_j^{-,r}\}$ and $\{o_j^{+,r}\}$ can be computed **approximately** by probing.

Deriving upper bounds for $\{o_j^{-,r}\}$ and $\{o_j^{+,r}\}$ by probing

Let ℓ^0 and u^0 (respectively, ℓ^1 and u^1) be the lower and upper bounds of variables x , obtained by probing on $x_k = 0$ (respectively, $x_k = 1$). Then it follows that

$$\begin{aligned} o_j^{-,r} &= \max_{x_j > \ell_j} \left\{ \sum_{s \in \mathcal{N} \setminus \{j\}} a_{rs} x_s \mid x \in \mathcal{X} \right\} \\ &\leq \max \left\{ \sum_{\substack{s \in \mathcal{N} \setminus \{j\}, \\ a_{rs} > 0}} a_{rs} u_s^0 + \sum_{\substack{s \in \mathcal{N} \setminus \{j\}, \\ a_{rs} < 0}} a_{rs} \ell_s^0, \sum_{\substack{s \in \mathcal{N} \setminus \{j\}, \\ a_{rs} > 0}} a_{rs} u_s^1 + \sum_{\substack{s \in \mathcal{N} \setminus \{j\}, \\ a_{rs} < 0}} a_{rs} \ell_s^1 \right\}. \end{aligned} \quad (8)$$

► When $j = k$, (8) is reduced to

$$o_k^{-,r} \leq \sum_{\substack{s \in \mathcal{N} \setminus \{j\}, \\ a_{rs} > 0}} a_{rs} u_s^1 + \sum_{\substack{s \in \mathcal{N} \setminus \{j\}, \\ a_{rs} < 0}} a_{rs} \ell_s^1. \quad (9)$$

► Given a probing variable x_k , complexity of computing $o_j^{-,r}$, $\forall j \in \mathcal{N}$ and $r \in \mathcal{M}$ with $a_{rj} < 0$ is $|\mathcal{N}| \times \text{nnz}$.

Deriving upper bounds for $\{o_j^{-,r}\}$ and $\{o_j^{+,r}\}$ by probing

The complexity of $|\mathcal{N}| \times \text{nnz}$ can be reduced. First, we compute

$$\alpha_r^0 = \sum_{s \in \mathcal{N}, a_{rs} > 0} a_{rs} u_s^0 + \sum_{s \in \mathcal{N}, a_{rs} < 0} a_{rs} \ell_s^0 \text{ and } \alpha_r^1 = \sum_{s \in \mathcal{N}, a_{rs} > 0} a_{rs} u_s^1 + \sum_{s \in \mathcal{N}, a_{rs} < 0} a_{rs} \ell_s^1 \quad (10)$$

for all $r \in \mathcal{M}$ with the complexity of $\mathcal{O}(\text{nnz})$. Then, using

$$\sum_{\substack{s \in \mathcal{N} \setminus \{j\}, \\ a_{rs} > 0}} a_{rs} u_s^0 + \sum_{\substack{s \in \mathcal{N} \setminus \{j\}, \\ a_{rs} < 0}} a_{rs} \ell_s^0 = \alpha_r^0 - a_{rj} \ell_j^0, \quad (11)$$

$$\sum_{\substack{s \in \mathcal{N} \setminus \{j\}, \\ a_{rs} > 0}} a_{rs} u_s^1 + \sum_{\substack{s \in \mathcal{N} \setminus \{j\}, \\ a_{rs} < 0}} a_{rs} \ell_s^1 = \alpha_r^1 - a_{rj} \ell_j^1, \quad (12)$$

Deriving upper bounds for $\{o_j^{-,r}\}$ and $\{o_j^{+,r}\}$ by probing

The right-hand side of (8) reduces to

$$\begin{aligned} o_j^{-,r} &\leq \max \left\{ \sum_{\substack{s \in \mathcal{N} \setminus \{j\}, \\ a_{rs} > 0}} a_{rs} u_s^0 + \sum_{\substack{s \in \mathcal{N} \setminus \{j\}, \\ a_{rs} < 0}} a_{rs} \ell_s^0, \sum_{\substack{s \in \mathcal{N} \setminus \{j\}, \\ a_{rs} > 0}} a_{rs} u_s^1 + \sum_{\substack{s \in \mathcal{N} \setminus \{j\}, \\ a_{rs} < 0}} a_{rs} \ell_s^1 \right\} \\ &= \max \{ \alpha_r^0 - a_{rj} \ell_j^0, \alpha_r^1 - a_{rj} \ell_j^1 \}, \end{aligned}$$

and can be computed in **constant time**. As a result, the upper bounds of $o_j^{-,r}$ for all $j \in \mathcal{N}$ and $r \in \mathcal{M}$ with $a_{rj} < 0$ can be conducted in $\mathcal{O}(\mathbf{nnz})$.

CONTENTS

- ① Preliminary
- ② Integration of dual fixing into probing
- ③ Improved conditions for applying dual fixing
- ④ Numerical experiments
- ⑤ Conclusions

Environment and settings

Environment

- ▶ Hardware: a cluster of Intel(R) Xeon(R) Gold 6140R CPU @ 2.30GHz.
- ▶ MIP solver: [HiGHS](#) 1.7.2.
- ▶ Time limit: 7200 seconds.
- ▶ Test set: MIPLIB 2017 benchmark 240 instances, with five random seeds.

Settings

- ▶ Default: [HiGHS](#) default parameter.
- ▶ DF+Probing: Default + Dual fixing augmented probing.
- ▶ GDF: Default + Improved conditions for dual fixing.
- ▶ All: DF+Probing + GDF

Default v.s. DF+Probing

Instances are included in " $\geq x$ " bracket if it is solved by at least one solver, and the slower of the two uses $\geq x$ seconds.

Table 1: Performance impact of embedding dual fixing into the probing framework.

	Default				DF+Probing			Compare		Affected		
Bracket	#Ins	#S	T	N	#S	T	N	T	N	#Ins	T	N
≥ 0	740	732	279.32	849.94	731	269.53	812.25	0.96	0.96	238	0.89	0.86
≥ 10	673	665	429.65	1457.07	664	413.09	1386.05	0.96	0.95	232	0.89	0.86
≥ 100	510	502	951.34	4059.32	501	898.31	3801.08	0.94	0.94	206	0.86	0.84
≥ 1000	268	260	2682.28	15240.80	259	2448.18	13536.65	0.91	0.89	126	0.82	0.76

Default v.s. DF+Probing

To better understand where the improvements come from, we further compare the **total number of bound changes** derived in the first round of probing.

More bound changes $\Rightarrow$ More reductions are detected

Table 2: Comparison of the bound changes detected in the first round of probing with and without dual fixing embedded.

Bracket	Default		DF+Probing		Compare		Affected	
	#Bchg	T _{probing}	#Bchg	T _{probing}	#Bchg	T _{probing}	#Bchg	T _{probing}
≥ 0	2206.26	0.47	2616.32	0.51	1.19	1.09	1.65	1.12
≥ 10	2342.51	0.50	2824.05	0.55	1.21	1.09	1.67	1.12
≥ 100	2788.56	0.55	3446.58	0.59	1.24	1.08	1.64	1.09
≥ 1000	2224.87	0.57	2979.63	0.63	1.34	1.10	1.83	1.15

Default v.s. GDF

Table 3: Performance impact of using probing to detect (generalized) dual fixings

	Default				GDF			Compare		Affected		
Bracket	#Ins	#S	T	N	#S	T	N	T	N	#Ins	T	N
≥ 0	735	732	273.20	879.37	734	272.79	872.41	1.00	0.99	75	0.96	0.92
≥ 10	668	665	420.66	1502.90	667	419.20	1489.80	1.00	0.99	72	0.95	0.92
≥ 100	500	497	955.22	4419.01	499	946.60	4367.56	0.99	0.99	72	0.95	0.92
≥ 1000	258	255	2709.47	16216.82	257	2629.56	15682.15	0.97	0.97	48	0.87	0.82

Default v.s. All

Table 4: Performance impact of embedding dual fixing into the probing framework, and using probing to detect (generalized) dual fixings

	Default				All			Compare		Affected		
Bracket	#Ins	#S	T	N	#S	T	N	T	N	#Ins	T	N
≥ 0	740	732	279.32	847.30	730	269.32	800.95	0.96	0.95	267	0.89	0.85
≥ 10	673	665	429.65	1453.28	663	412.14	1365.93	0.96	0.94	261	0.89	0.85
≥ 100	509	501	955.66	4117.09	499	897.15	3793.28	0.94	0.92	235	0.87	0.83
≥ 1000	268	260	2682.28	15280.14	258	2414.31	13293.01	0.90	0.87	146	0.82	0.76

CONTENTS

- 1 Preliminary
- 2 Integration of dual fixing into probing
- 3 Improved conditions for applying dual fixing
- 4 Numerical experiments
- 5 Conclusions

Our contributions

- ▶ We developed two methods to **combine and enhance** these two powerful techniques for MIP problems.
- ▶ Through computational experiments using the open-source solver HiGHS, we demonstrate that with the two proposed methods, a **4%** decrease in solution time and a **5%** decrease in number of nodes can be achieved over the MIPLIB 2017 benchmark instances.

References I

- Achterberg, T. (2007). *Constraint Integer Programming*. PhD thesis, Technische Universitat Berlin.
- Achterberg, T., Bixby, R. E., Gu, Z., Rothberg, E., and Weninger, D. (2019). Presolve reductions in mixed integer programming. *INFORMS Journal on Computing*, 32(2):473–506.
- Achterberg, T. and Wunderling, R. (2013). Mixed integer programming: analyzing 12 years of progress. In *Facets of Combinatorial Optimization: Festschrift for Martin Grötschel*, pages 449–481. Springer Berlin Heidelberg.
- Gleixner, A., Gottwald, L., and Hoen, A. (2023). PAPILO: a parallel presolving library for integer and linear optimization with multiprecision support. *INFORMS Journal on Computing*, 35(6):1329–1341.
- Mészáros, C. and Suhl, U. H. (2003). Advanced preprocessing techniques for linear and quadratic programming. *OR Spectrum*, 25(4):575–595.
- Savelsbergh, M. W. P. (1994). Preprocessing and probing techniques for mixed integer programming problems. *ORSA Journal on Computing*, 6(4):445–454.

Thank you all for your attention!